6.3 编写自定义Skill

> **生成模型**：OpenAI GPT-5.3 Codex（openai/gpt-5.3-codex）
> **Token 消耗**：输入约 13k，输出约 6.5k（本节，估算值）

前两节我们做了两件事：

- 搞清楚 Skill 是什么；
- 把常用 Skills 跑起来了。

现在进入最关键的一步：**你自己写一个 Skill**。

很多同学会觉得“自定义 Skill”听起来像插件开发，门槛很高。其实第一版完全可以很轻：

- 不写复杂运行时代码；
- 先把团队常见流程写成 `SKILL.md` ；
- 在项目里反复实战，迭代内容。

你把它理解成“把你脑子里的经验，变成 Agent 可稳定复现的操作手册”。

6.3.1 先明确目标：Skill 解决的是“重复决策”

一个好 Skill 通常有这三个特征：

1. **场景高频**：每周都会遇到；
2. **步骤固定**：有比较清晰的 SOP；
3. **错误成本高**：做错会带来明显损失。

举几个很适合做成 Skill 的题目：

- 发布前检查（版本号、变更日志、测试结果）；
- 值班排障（先看日志、再看告警、再出结论）；
- PR 质检（模板检查、风险分级、回滚方案）。

如果一个任务每次都“随缘操作”，不稳定，那就值得 Skill 化。

6.3.2 SKILL.md 的核心结构（你至少要掌握这些字段）

基于前面章节的规范， `SKILL.md` 一般由两部分组成：

1. YAML Front Matter（元数据）
2. 正文（给模型的执行说明）

最小可用模板：

```
---
name: release-helper
description: 帮助执行发布前检查、版本发布与发布后验证。
---

# Release Helper

当用户要求发布时：
1. 读取 package.json 和 CHANGELOG.md。
2. 检查测试与构建状态。
3. 生成发布说明并提示风险。
```

常见字段说明（实战够用版）

- `name` ： 技能唯一标识，尽量简洁稳定；
- `description` ： 一句话描述用途，建议控制在短句；
- `homepage` ： 可选，放文档地址；
- `user-invocable` ： 可选，是否允许用户显式调用；
- `disable-model-invocation` ： 可选，是否禁止自动注入；
- `command-dispatch` / `command-tool` ： 可选，用于确定性命令分发；
- `metadata` ： 可选，扩展信息（如依赖命令、emoji、安装提示）。

> **衍生解释：Front Matter 是什么？**
>
> 这是 Markdown 常见的“头部元数据区”，本质是结构化配置。你可以把它看成“文档的配置文件嵌在文档里”。在 Skill 里，它负责让系统快速识别能力属性，而正文负责给模型行为指引。

6.3.3 从 0 开始写一个 Skill：完整步骤

我们来做一个可直接上手的例子： `release-helper` 。

目标：当你说“准备发版”时，Agent 能按固定流程完成检查并给出操作建议。

Step 1：创建目录

```
mkdir -p ./skills/release-helper
```

为什么放 `./skills` ？因为这是 workspace 级技能目录，最适合开发调试，改完就能在当前项目验证。

Step 2：创建 SKILL.md

```
---
name: release-helper
description: 面向 Node 项目的发布助手，执行发版前检查并生成发布说明。
metadata:
  {
    "openclaw":
      {
        "emoji": "🚀",
        "requires": { "bins": ["git", "node", "npm"] }
      }
  }
---

# Release Helper

## 适用场景

- 用户希望发布新版本（beta / rc / stable）
- 需要生成变更摘要与风险提示

## 执行原则

1. 先检查，再发布；
2. 每一步都给出可验证证据；
3. 遇到失败立即停止并报告。

## 建议流程

1. 读取 `package.json`、`CHANGELOG.md`。
2. 执行测试与构建（例如 `npm test`、`npm run build`）。
3. 检查 Git 工作区是否干净。
4. 生成版本变更摘要（新增 / 修复 / 破坏性变更）。
5. 给出发布命令建议与回滚提示。

## 输出格式

- 发布准备状态：通过 / 失败
- 关键证据：测试、构建、工作区状态
- 建议动作：下一步命令
- 风险提示：高优先级风险项
```

这份内容已经是可用的第一版。

Step 3：检查技能可见性

```
openclaw skills list
openclaw skills info release-helper
openclaw skills list --eligible
```

如果能看到 `release-helper` ，说明目录和文件格式基本正确。

Step 4：对话验证

你可以直接发：

```
请按 release-helper 流程检查当前项目是否可以发版，并给出建议版本号。
```

观察点有三个：

1. 是否按你写的步骤执行；
2. 输出结构是否符合你定义的格式；
3. 遇到失败是否会停止而不是“硬往下走”。

6.3.4 写 Skill 时最容易忽略的四个设计点

1）边界条件

不要只写“理想路径”，要写失败路径。

比如：

- `CHANGELOG.md` 不存在怎么办；
- 测试命令失败怎么办；
- Git 有未提交改动怎么办。

你可以在 Skill 里明确：

```
若测试失败，停止后续步骤，并输出失败测试摘要与建议修复路径。
```

2）输出结构

很多 Skill 失效感强，是因为输出不稳定。

建议你给出固定骨架，例如：

- 结论；
- 证据；
- 下一步；
- 风险。

模型会更容易稳定复现。

3）可执行性

Skill 里写的命令必须能在你团队环境跑起来，不要写“理论命令”。

4）最小可维护

第一版别贪大。先把一个流程做稳，再扩展。

6.3.5 调试方法：从“看不到”到“跑不对”逐层排

调试 Skill 你可以按三层来。

第一层：识别层（系统能不能识别）

```
openclaw skills list
openclaw skills info release-helper
```

如果找不到，通常是：

- 目录层级不对；
- 文件名不是 `SKILL.md` ；
- Front Matter 语法错误。

第二层：资格层（环境是否满足）

```
openclaw skills list --eligible
openclaw skills check
```

如果不 eligible，多半是依赖命令不存在或环境变量没注入。

第三层：行为层（执行是否符合设计）

同一个提示跑三次，观察步骤是否稳定；
如果不稳定，就把 Skill 正文写得更明确，减少模糊词。

例如把“适当检查”改成“按顺序执行 A、B、C，任一步失败立刻停止”。

6.3.6 给 Skill 加配置：让它适配不同项目

你不希望每个项目都复制一个版本，可以通过配置抽象参数。

示例：

```jsonc
{
  skills: {
    entries: {
      "release-helper": {
        enabled: true,
        env: {
          RELEASE_CHANNEL: "$RELEASE_CHANNEL"
        },
        config: {
          testCommand: "npm test",
          buildCommand: "npm run build",
          changelogPath: "CHANGELOG.md",
          requireCleanGit: true
        }
      }
    }
  }
}
```

这样同一个 Skill 能在不同仓库里复用，只改配置不改正文。

6.3.7 进阶：让 Skill 更“像团队成员”

当你的 Skill 能稳定跑后，可以继续增强三件事：

1. **术语统一**：和团队文档一致（例如“阻塞项”“可回滚”）；
2. **模板统一**：输出对齐你们 PR/Issue 模板；
3. **安全约束**：明确不可执行高风险动作（如未经确认的破坏性命令）。

你会发现，一个成熟 Skill 不只是“会做事”，还是“按团队方式做事”。

6.3.8 发布到 ClawHub：从“本地私用”到“生态共享”

当一个 Skill 在团队里跑稳，你可以考虑发布到 ClawHub，让更多人安装使用。

典型流程可以概括为：

1. 整理目录与文档（至少有清晰 `SKILL.md` ）；
2. 补充依赖说明和使用示例；
3. 本地安装验证（干净环境复测）；
4. 提交到 ClawHub（按其发布规范）。

你可以先做“私有发布”，再逐步开放。

发布前检查清单（建议直接复用）

- 名称是否清晰、无冲突风险；
- 描述是否 1 句话讲清用途；
- 是否写明依赖命令与环境变量；
- 是否有最少 3 个真实使用示例；
- 是否写了失败处理策略；
- 是否有版本更新记录。

> **衍生解释：为什么“发布前检查”这么重要？**

> Skill 一旦进入生态，就不只是你自己用。它变成别人工作流的一部分。如果说明不完整，别人会在生产场景踩坑。开源生态里“可读性和可维护性”本身就是产品质量。

6.3.9 一个可直接复制的 SKILL.md 模板

下面给一个你可以直接改名使用的模板：

```
---
name: your-skill-name
description: 一句话描述这个技能解决的问题。
homepage: https://example.com/your-skill-docs
metadata:
  {
    "openclaw":
      {
        "emoji": "🧩",
        "requires": { "bins": ["git"] }
      }
  }
---

# Your Skill Name

## 适用场景

- 场景 A
- 场景 B

## 执行步骤

1. 先做什么。
2. 再做什么。
3. 失败时怎么处理。

## 输出要求

- 结论
- 证据
- 下一步
- 风险

## 示例请求

1. 示例请求一。
2. 示例请求二。
```

你会发现，好的 Skill 模板其实和写技术文档很像：边界清晰、步骤明确、输出可验证。

6.3.10 本节实操任务（建议你立刻做）

为了把知识变成手感，建议你现在就做这个 30 分钟练习：

1. 在当前项目创建 `./skills/pr-audit/SKILL.md` ；
2. 让它专门负责 PR 审查：风险分级 + 回滚建议；
3. 用真实仓库分支验证一次；
4. 记录你改了哪三处提示词让结果更稳定。

这个练习做完，你就真的跨过“会看教程”到“会写 Skill”这条线了。

本节小结

这一节的核心不是语法，而是方法：

- 自定义 Skill 的本质，是把重复决策沉淀成可复用流程；
- `SKILL.md` 由元数据 + 行为说明组成，先做最小可用版最稳；
- 开发路径建议走 workspace：创建 → 识别 → eligible → 对话验证；
- 调试要分三层：识别层、资格层、行为层；
- 发布到 ClawHub 前，先把依赖、示例、失败路径写完整。

到这里，第 6 章的目标就达成了：你不但会用 Skills 生态，也有能力写出自己的第一个可运行 Skill。接下来再读第 29 章，你会从“知道概念”升级到“理解系统实现”。